



객체지향 프로그램의 시작

- 객체지향 이전의 프로그래밍
- 객체지향의 등장 배경과 이해
- 메소드의 이해

6-1. 객체지향 이전의 프로그래밍

절차지향

지금까지 배운 내용으로 1부터 10까지의 정수의 합을 구하는 프로그램을 만들어 보자.

그리고, 그 결과값이 짝수 인지 홀수 인지 파악하고, 홀수라면 '홀수 입니다.'를 출력하고, 짝수라면 '짝수 입니다.'를 출력해 보자.

(6_1_ex1_proceEx)

```
ProceEx.java
1 package com.javalec.proceEx;
2
3 public class ProceEx {
4     public static void main(String[] args) {
5
6         int sum = 0;
7         int result = 0;
8
9         for (int i = 0; i <= 10; i++) {
10             sum = sum + i;
11         }
12         System.out.println("1부터 10까지의 정수의 합은 " + sum + "입니다.");
13
14         result = sum % 2;
15         if(result == 0){
16             System.out.println("짝수 입니다.");
17         } else {
18             System.out.println("홀수 입니다.");
19         }
20     }
21 }
22
23
```

```
<terminated> ProceEx [Java Appl
1부터 10까지의 정수의 합은 55입니다.
홀수 입니다.
```

6-1. 객체지향 이전의 프로그래밍

절차지향

위에서 살펴본 프로그래밍 방식이 절차지향 언어 입니다.

위에서부터 쭉~~~~~ 내려오면서 실행 됩니다.

절차지향 방식을 이용하면서 개발자들은 몇 가지 비효율적인 면을 느끼게 됩니다.

첫째, 기존의 1부터 10까지의 합을 구하는 것을 5부터 100까지로 변경하게 되면 동일한 작업이지만 다시 코드를 수정해야 됩니다.

둘째, 기존의 로직을 다시 사용하게 될 때, 다시 코딩을 하는 방법과, 기존 코드를 카피해서 재사용 하는 방식이 있으나, 두 방법 모두 비효율적 입니다.

셋째, 두번째 방식으로 코딩을 하면 동일한 로직의 코드가 문서 내에 넘쳐나, 코드의 양이 너무 길어 집니다. 추후에 유지보수가 어렵습니다.

하여, 등장한 방식이 함수 또는 **메소드**라고 불리는 녀석 입니다. **로직만 만들어 놓고, 그때 그때 소스(데이터)를 주면 메소드가 알아서 결과값을 반환하는 방식** 입니다.

위의 코드를 수정해 보도록 하겠습니다. (6_1_ex2_proceEx)

```
public int sum(int i, int j) {  
    // TODO Auto-generated method stub  
  
    int r = 0;  
  
    for (int h = i; h <= j; h++) {  
        r = r + h;  
    }  
  
    return r;  
}
```

```
public String evenOdd(int sum) {  
    // TODO Auto-generated method stub  
    String r = new String();  
  
    if((sum % 2) == 0){  
        r = "짝수 입니다.";  
    } else {  
        r = "홀수 입니다.";  
    }  
  
    return r;  
}
```

6-2. 객체지향의 등장 배경과 이해

객체지향의 등장 배경과 이해

메소드를 이용한 방식으로 개발자들은 아주 많은 효율성을 느끼게 되어 있습니다.

하지만, 메소드를 이용한 방식에도 한계는 있었습니다.

데이터가 많아 지고, 메소드가 많아짐에 따라 코드의 양이 많이 지고, 유지보수가 어려워 지기 시작 했습니다.

하여, 등장한 방식이 객체지향 프로그래밍 입니다.

객체란 동일한 성질의 데이터와 메소드를 한곳에 모아두고 필요한 곳에서 언제든지 이용할 수 있게 만들어 놓은 덩어리 입니다.

쉽게 생각해서 세상에 존재하는 모든 것을 객체라고 생각할 수 있으며, 이러한 객체를 이용한 프로그래밍 방식이 객체지향 방식 입니다.

예) 자동차, 자전거, 주방장,프린터기, 자판기, 휴대폰 등등.....

6-3. 메소드의 이해

메소드의 이해

```
public int sum(int i, int j) {  
  
    int r = 0;  
  
    for (int h = i; h <= j; h++) {  
        r = r + h;  
    }  
  
    return r;  
}
```

6-3. 객체지향의 등장 배경과 이해

메소드의 이해

예 : 사용자로부터 1에서 9까지 숫자 중 한 개의 숫자를 입력 받아 구구단을 출력해 보자.(6_3_ex1_methodEx)



객체지향 프로그램의 본질

- 메소드 프로그래밍의 한계
- 객체의 개념 및 클래스의 이해
- 객체지향 프로그래밍에서 데이터 처리 방법
- 객체자료형과 변수와의 관계
- 객체자료형과 데이터와의 관계

7-1. 메소드 프로그래밍의 한계

메소드 프로그래밍의 한계

메소드를 이용하면 로직의 재사용이 가능하여 개발을 효율적으로 할 수 있었습니다.

하지만, 메소드만 가지고는 많은 양의 로직을 처리하기에는 한계가 있습니다.

한 문서 내에 메소드의 수가 많아질 경우 추후 유지 보수에 많은 어려움이 발생합니다.

7-2. 객체의 개념 및 클래스의 이해

객체의 개념

먼저 시간에 객체의 의미에 대해서 간단하게 살펴 보았습니다.

다시 한번 살펴보면 **객체는 같은 종류의 데이터와 로직이 함께 있는 구성체** 입니다. 그리고 현실세계와 비교해 보면 세상의 모든 것이 객체 입니다.

예를 들어, 자동차 라는 객체가 있습니다. 자동차에는 운전대라는 메소드와 브레이크, 기어 라는 메소드가 있습니다. 그리고 색상, 배기량 등의 데이터가 있습니다.

자동차를 운전하는 사람은 운전대를 오른쪽, 왼쪽으로 돌려가며 운전하고, 브레이크를 밟아 정지하고, 기어를 작동하여 앞,뒤로 운전 합니다.

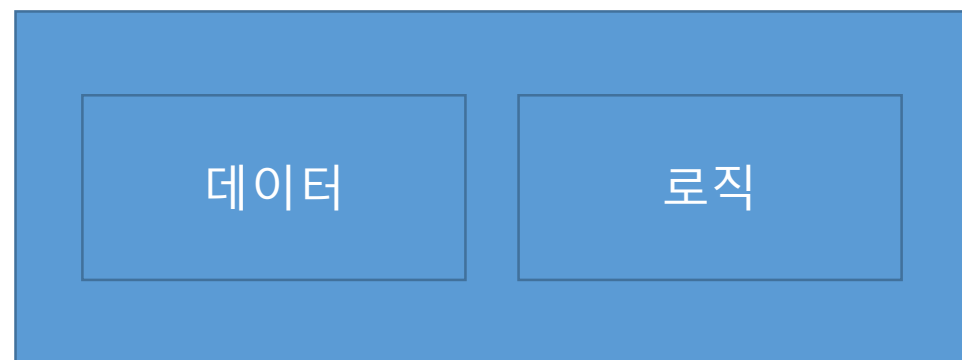
또한 색상과 배기량을 가지고 다른 자동차와 구분하기도 합니다.

이처럼 객체는 메소드와 비슷하지만 그 상위 개념이라고 할 수 있습니다. 즉 **완전한 독립체라고** 할 수 있습니다.

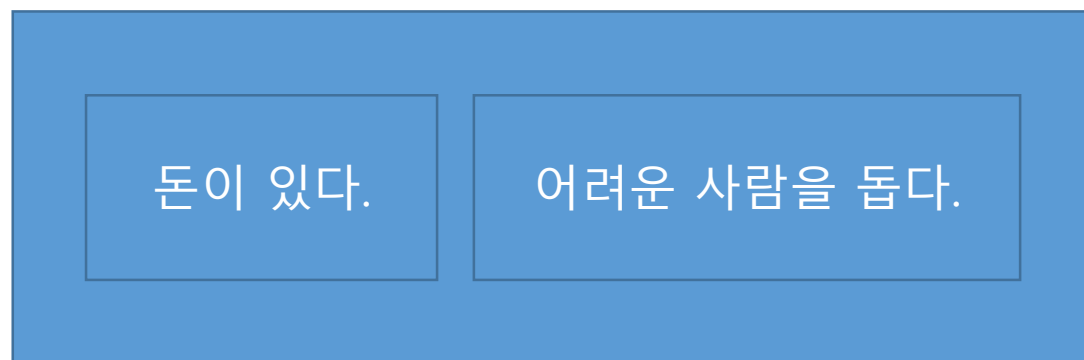
객체는 클래스로부터 생산(복사본)됩니다.

예를 들어, 그랜저 자동차가 공장에서 생산됩니다. 이때 색상이 정해지지 않은 자동차를 만들어 놓고, 소비자의 요구에 따라서 색상을 칠합니다.

즉 색을 칠하기 전의 자동차가 클래스이며, 소비자(개발자)의 요구에 따라서 색을 칠해서 완전한 자동차(객체)가 만들어 집니다.



프로그램의 객체



현실 세계의 객체(돈이 많고, 착한 사람)

7-3. 객체지향 프로그래밍에서 데이터 처리 방법

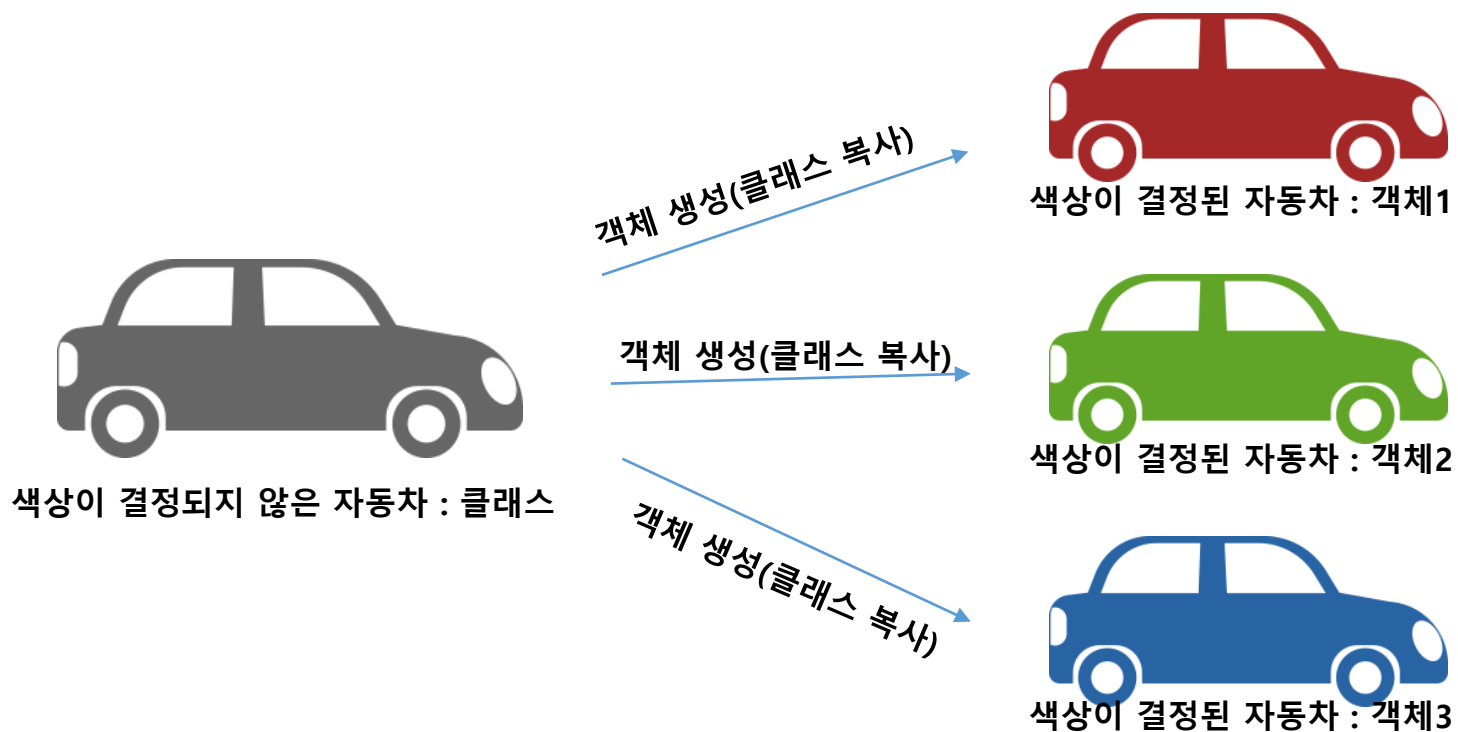
데이터 처리 방법

그랜저 자동차 얘기를 계속 하겠습니다. 색상이 결정 되지 않은 자동차를 클래스라고 한다고 했습니다.

그리고 구매자가 색상을 결정하여 계약을 하면, 그때 클래스로 부터 객체가 생성 됩니다. 그리고 생성된 객체는 계약을 한 소비자의 것 입니다.

또 다른 소비자가 다른 색상으로 계약을 하면 또 하나의 객체가 생성됩니다.

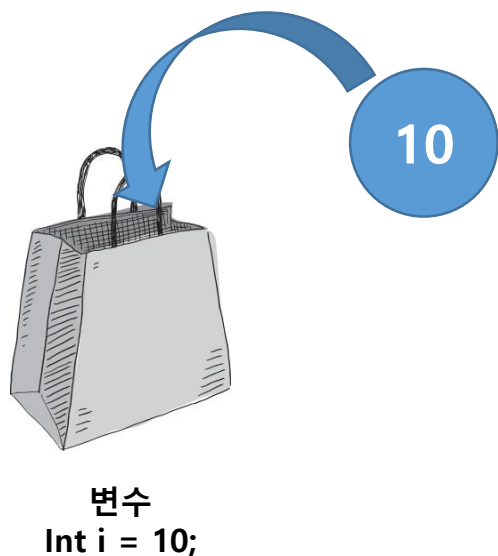
그림으로 살펴보면 쉽게 이해 될 수 있습니다.



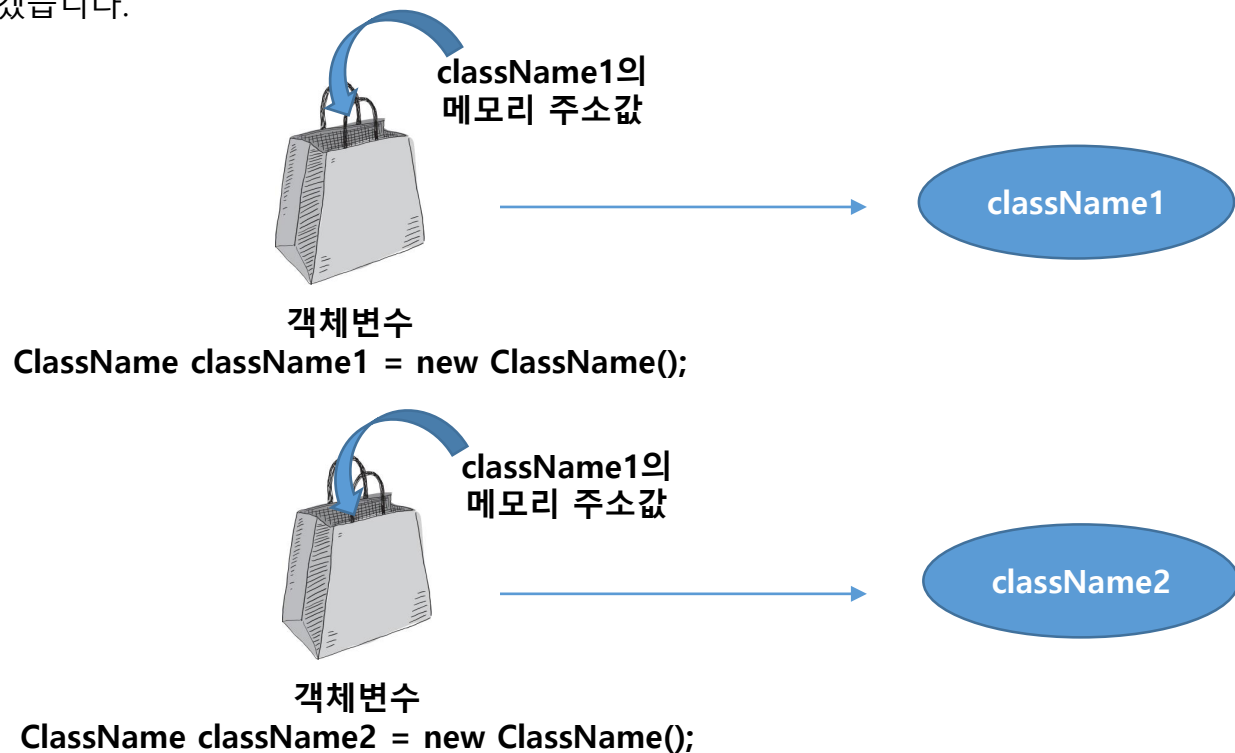
7-4. 객체 자료형과 변수와의 관계

변수(기본 데이터)와 객체의 메모리 구조

기본 데이터 변수는 메모리상에 무엇이든지 담을 수 있는 주머니를 만들어 놓고 주머니에 값을 직접 넣는다고 배웠습니다. 그럼 객체 자료형은 메모리상에 어떻게 존재 하는지 알아 보도록 하겠습니다.



기본 자료형



객체 자료형

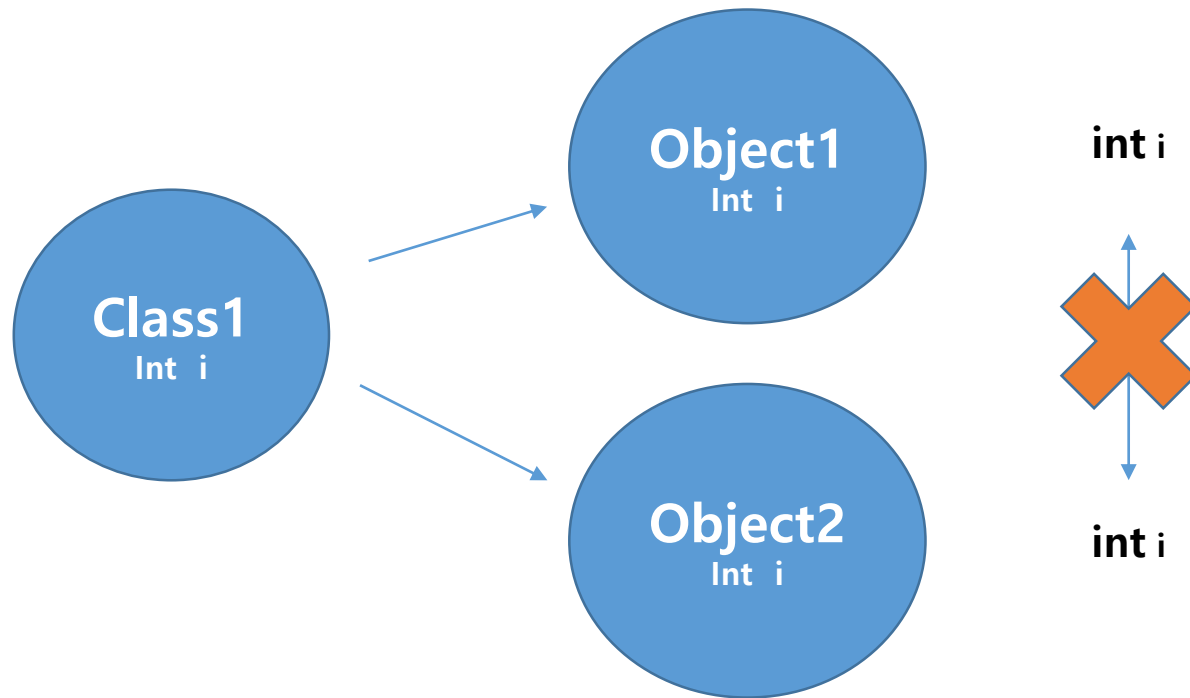
7-5. 객체 자료형과 데이터와의 관계

객체 안의 데이터

객체는 클래스로부터 생성됩니다.

그리고, 생성된 객체는 동일한 클래스에서 생성되었다 하더라도 이제는 완전한 **독립체** 입니다.

따라서 객체 안에 종속되어 있는 데이터는 완전히 별개의 데이터입니다.



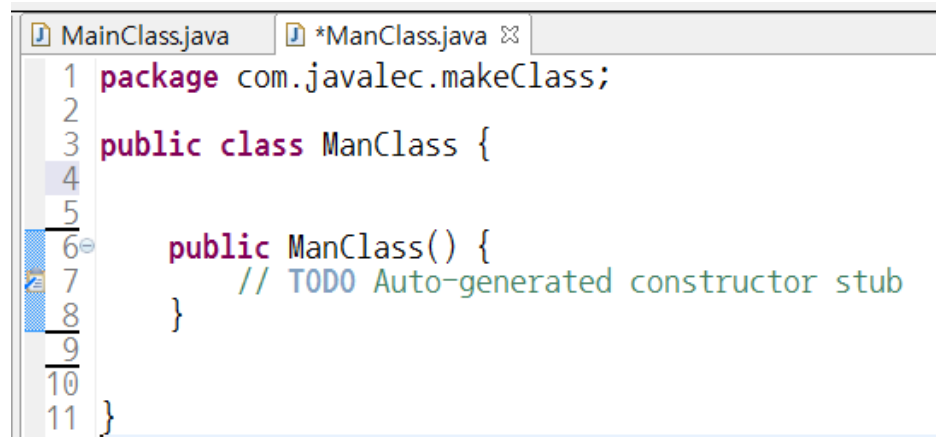


객체지향 프로그램의 기초적인 코딩 방법

- 클래스 제작
- 메소드와 인스턴스변수 만들기
- 생성자의 이해
- this 키워드의 이해

8-1. 클래스 제작

클래스를 만들어 보자(8_1_ex1_makeClass)



```
1 package com.javalec.makeClass;
2
3 public class ManClass {
4
5
6     public ManClass() {
7         // TODO Auto-generated constructor stub
8     }
9
10
11 }
```

1. 패키지명
2. 클래스명
3. 생성자

8-2. 메소드와 인스턴스변수 만들기

메소드, 인스턴스 변수 만들기(getter, setter)(8_1_ex1_makeClass)

```
1 package com.javalec.makeClass;
2
3 public class ManClass {
4
5     private int age;
6     private int height;
7     private int weight;
8     private String phoneNum;
9
10    public ManClass() {
11        // TODO Auto-generated constructor stub
12    }
13
14    public double calculateBMI() {
15        // TODO Auto-generated method stub
16        double result = weight / (height * height);
17        return result;
18    }
19
20    public int getAge() {
21        return age;
22    }
23
24    public void setAge(int age) {
25        this.age = age;
26    }
```

1. 인스턴스 변수(멤버변수)
2. 메소드
3. Getter & setter

8-3. 생성자의 이해

생성자 (8_1_ex1_makeClass)

```
private int age;
private int height;
private int weight;
private String phoneNum;

public ManClass() {
    // TODO Auto-generated constructor stub
}

public ManClass(int age, int height, int weight, String phoneNum) {
    // TODO Auto-generated constructor stub
    this.age = age;
    this.height = height;
    this.weight = weight;
    this.phoneNum = phoneNum;
}
```


8-4. this 키워드의 이해

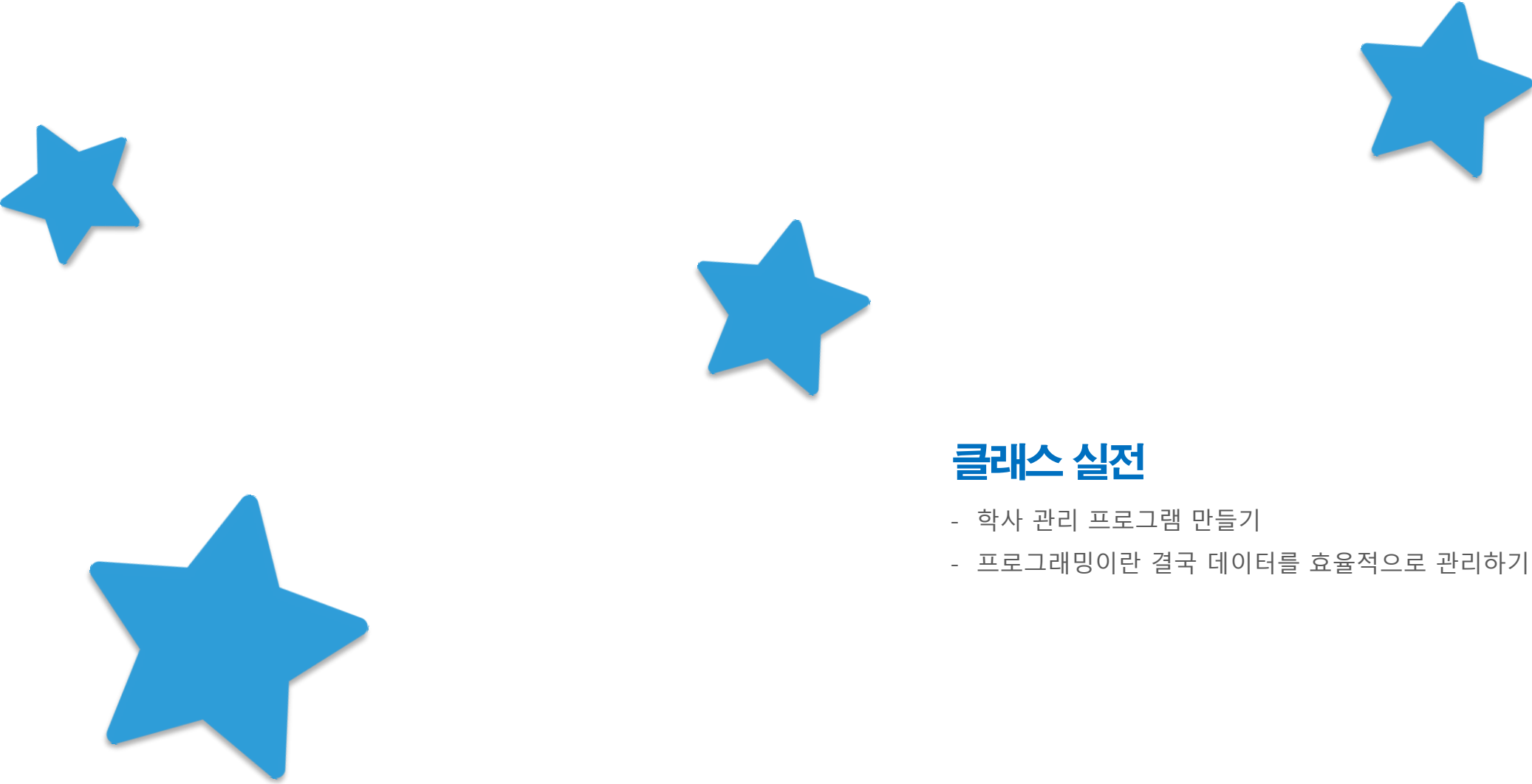
This키워드 (8_1_ex1_makeClass)

```
private int age;
private int height;
private int weight;
private String phoneNum;

public ManClass() {
    // TODO Auto-generated constructor stub
}

public ManClass(int age, int height, int weight, String phoneNum) {
    // TODO Auto-generated constructor stub
    this.age = age;
    this.height = height;
    this.weight = weight;
    this.phoneNum = phoneNum;
}
```

```
public int getAge() {
    // return age;
    return this.age;
}
```



클래스 실전

- 학사 관리 프로그램 만들기
- 프로그래밍이란 결국 데이터를 효율적으로 관리하기 위한 도구

9-1. 학사 관리 프로그램 만들기

학사 관리 프로그램(9_1_ex1_studentmanager)

필요한 클래스들을 머릿속으로 생각해 보자. (추상화 즉 모델링과정)

우선 main 메소드가 있는 학사관리 시스템 클래스가 있어야 할 거 같습니다.

다음으로 학사관리는 학생을 관리하는 시스템이므로 학생에 대한 클래스가 있으면 좋을 거 같습니다.

그리고 학생에 대한 클래스에는 학생의 이름, 나이, 학번, 전공, 성별 등의 데이터가 포함되어 있어야 할 거 같습니다.

물론 데이터를 언제든지 업데이트 할 수 있어야 하겠습니다.

우울한 일이지만 재적 당한 학생에 대해서는 별도로 관리하는 클래스를 하나 추가해 보기로 합니다.

학사관리 시스템 클래스
StudentManager

학생 정보 클래스
Student

재적 학생 클래스
StudentExpel

9-1. 학사 관리 프로그램 만들기

학사 관리 프로그램(9_1_ex1_studentmanager)

```
StudentManager.java Student.java StudentExpel.java
1 package com.javalec.manager;
2
3 import java.util.ArrayList;
4
5 public class StudentManager {
6
7     ArrayList<Student> students = new ArrayList<Student>();
8     StudentExpel studentExpel = new StudentExpel();
9
10    public static void main(String[] args) {
11
12        StudentManager studentManager = new StudentManager();
13
14        studentManager.addStudent("홍길동", 22, 20123487, "공학부");
15        studentManager.addStudent("홍길준", 24, 20106428, "공학부");
16        studentManager.addStudent("이은경", 21, 20135788, "공학부");
17        studentManager.addStudent("김철수", 23, 20114221, "공학부");
18        studentManager.addStudent("김순희", 26, 20089883, "공학부");
19
20    }
21
22    private void addStudent(String name, int age, int studentNum, String major) {
23        // TODO Auto-generated method stub
24        students.add(new Student(name, age, studentNum, major));
25        System.out.println("학생 정보 입력 성공!!");
26    }
27 }
28
```

<terminated> StudentManager [Java Application] C:\Program Files\Java\jre1.8.0_25\bin\javaw.exe (2014. 12. 3. 오후 3:46:43)

학생 정보 입력 성공!!
학생 정보 입력 성공!!
학생 정보 입력 성공!!
학생 정보 입력 성공!!
학생 정보 입력 성공!!

9-2. 프로그래밍이란 결국 데이터를 효율적으로 관리하기 위한 도구

핵심은 데이터의 효율적인 관리

프로그래밍을 처음 접하는 친구들 또는 개발을 조금해본 친구들과 같이 일을 하다 보면, 프로젝트 시작과 동시에 이클립스를 열고 열심히 코딩을 시작 합니다. 물론 틀린 행동은 아닙니다. 하지만, 아주 긍정적인 행동도 아닙니다.

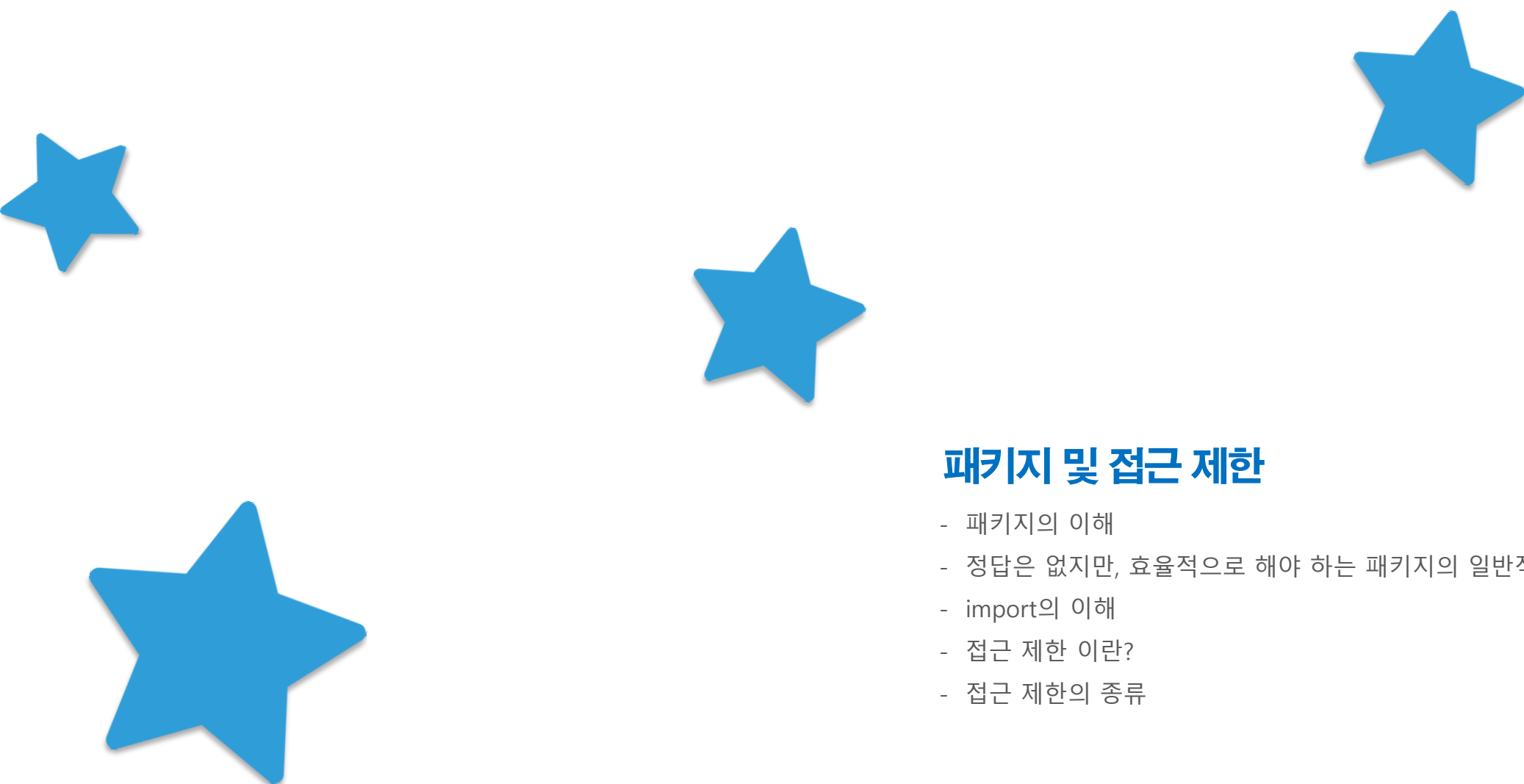
프로그래밍이란 버그 없이 돌아가야 합니다. 이것은 당연한 말 입니다.

여기에 우리가 꼭 생각해야 되는 것이 있습니다. 바로 프로그래밍이란 작업은 데이터를 누군가가 효율적으로 활용할 수 있게 시스템을 만드는 행동 입니다.

버그없이 돌아는 가는데, 데이터를 다루는 기술이 부족하여 시스템 속도가 너무 느리다면, 효율적인 시스템이라고 할 수 없을 것 입니다.

우리 개발자들은 코딩 보다 프로그래밍 구조에 대해서 더욱 깊이 생각해야 됩니다.

시간을 내어 리팩토링 관련 서적을 보는 것도 많은 도움이 될 것 입니다.



패키지 및 접근 제한

- 패키지의 이해
- 정답은 없지만, 효율적으로 해야 하는 패키지의 일반적인 설계 방식
- import의 이해
- 접근 제한이란?
- 접근 제한의 종류

10-1. 패키지의 이해

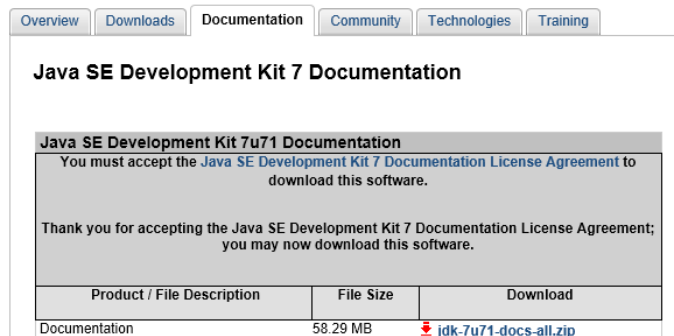
패키지란?

윈도우에는 윈도우탐색기(window+E)가 있습니다. 윈도우 탐색기는 컴퓨터에 보관되어 있는 파일을 관리하기 위해 폴더를 만들고, 그러한 폴더를 효율적으로 다루기 위해 존재 합니다. 즉 폴더 구조를 한눈에 볼 수 있도록 만들어진 프로그램 입니다.

JAVA에도 많은 .java파일들이 존재 합니다. 많은 정도가 예상하기도 힘들 정도로 많이 존재 합니다. 또한 우리가 프로그래밍을 하면서 계속적으로 .java파일을 만들 것 입니다. 이러한 많은 파일을 효율적으로 관리하기 위해 JAVA에서는 '패키지'라는 구조를 사용 합니다. 우선 JAVA에 존재하는 패키지를 살펴 보도록 하겠습니다.

아래 주소접속

<http://www.oracle.com/technetwork/java/javase/documentation/java-se-7-doc-download-435117.html>



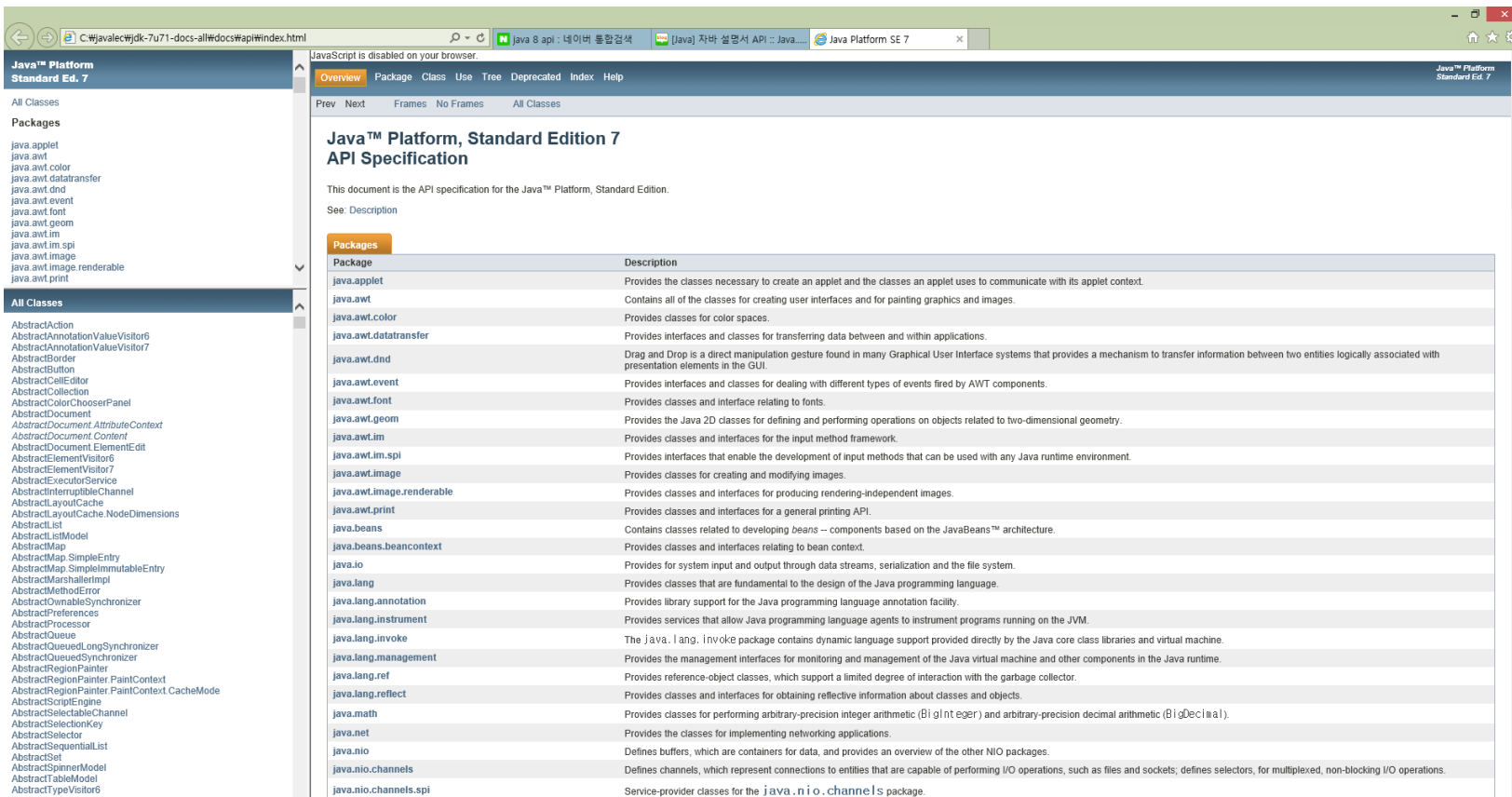
다운로드

C > 로컬 디스크 (C:) > javalec > jdk-7u71-docs-all > docs > api			
☐ 이름	수정한 날짜	유형	크기
index-files	2014-12-04 오후...	파일 폴더	
java	2014-12-04 오후...	파일 폴더	
javax	2014-12-04 오후...	파일 폴더	
org	2014-12-04 오후...	파일 폴더	
resources	2014-12-04 오후...	파일 폴더	
allclasses-frame.html	2014-09-26 오후...	HTML 문서	539KB
allclasses-noframe.html	2014-09-26 오후...	HTML 문서	461KB
constant-values.html	2014-09-26 오후...	HTML 문서	1,192KB
deprecated-list.html	2014-09-26 오후...	HTML 문서	143KB
help-doc.html	2014-09-26 오후...	HTML 문서	10KB
☒ index.html	2014-09-26 오후...	HTML 문서	3KB
overview-frame.html	2014-09-26 오후...	HTML 문서	23KB
overview-summary.html	2014-09-26 오후...	HTML 문서	63KB
overview-tree.html	2014-09-26 오후...	HTML 문서	1,313KB
package-list	2014-09-26 오후...	파일	5KB
serialized-form.html	2014-09-26 오후...	HTML 문서	1,345KB
stylesheet.css	2014-09-26 오후...	CSS 스타일시트 ...	11KB

10-1. 패키지의 이해

패키지란?

Java의 모든 클래스가 보입니다. 그리고 클래스를 관리하기 위한 패키지가 좌측상단에 보입니다.



JavaScript is disabled on your browser.

Overview Package Class Use Tree Deprecated Index Help

Prev Next Frames No Frames All Classes

Java™ Platform, Standard Edition 7 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.
See: Description

Package	Description
java.applet	Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
java.awt	Contains all of the classes for creating user interfaces and for painting graphics and images.
java.awt.color	Provides classes for color spaces.
java.awt.datatransfer	Provides interfaces and classes for transferring data between and within applications.
java.awt.dnd	Drag and Drop is a direct manipulation gesture found in many Graphical User Interface systems that provides a mechanism to transfer information between two entities logically associated with presentation elements in the GUI.
java.awt.event	Provides interfaces and classes for dealing with different types of events fired by AWT components.
java.awt.font	Provides classes and interface relating to fonts.
java.awt.geom	Provides the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.
java.awt.im	Provides classes and interfaces for the input method framework.
java.awt.im.spi	Provides interfaces that enable the development of input methods that can be used with any Java runtime environment.
java.awt.image	Provides classes for creating and modifying images.
java.awt.image.renderable	Provides classes and interfaces for producing rendering-independent images.
java.awt.print	Provides classes and interfaces for a general printing API.
java.beans	Contains classes related to developing beans – components based on the JavaBeans™ architecture.
java.beans.beancontext	Provides classes and interfaces relating to bean context.
java.io	Provides for system input and output through data streams, serialization and the file system.
java.lang	Provides classes that are fundamental to the design of the Java programming language.
java.lang.annotation	Provides library support for the Java programming language annotation facility.
java.lang.instrument	Provides services that allow Java programming language agents to instrument programs running on the JVM.
java.lang.invoke	The java.lang.invoke package contains dynamic language support provided directly by the Java core class libraries and virtual machine.
java.lang.management	Provides the management interfaces for monitoring and management of the Java virtual machine and other components in the Java runtime.
java.lang.ref	Provides reference-object classes, which support a limited degree of interaction with the garbage collector.
java.lang.reflect	Provides classes and interfaces for obtaining reflective information about classes and objects.
java.math	Provides classes for performing arbitrary-precision integer arithmetic (BigInteger) and arbitrary-precision decimal arithmetic (BigDecimal).
java.net	Provides the classes for implementing networking applications.
java.nio	Defines buffers, which are containers for data, and provides an overview of the other NIO packages.
java.nio.channels	Defines channels, which represent connections to entities that are capable of performing I/O operations, such as files and sockets; defines selectors, for multiplexed, non-blocking I/O operations.
java.nio.channels.spi	Service-provider classes for the java.nio.channels package.

10-1. 패키지의 이해

패키지란?

패키지는 클래스를 모아 놓은 폴더 입니다.

예를 들어 우리가 이전에 사용했던 Arrays 클래스를 보면 아래와 같습니다.



`java/util` 폴더에 있는 `Arrays`클래스 입니다.

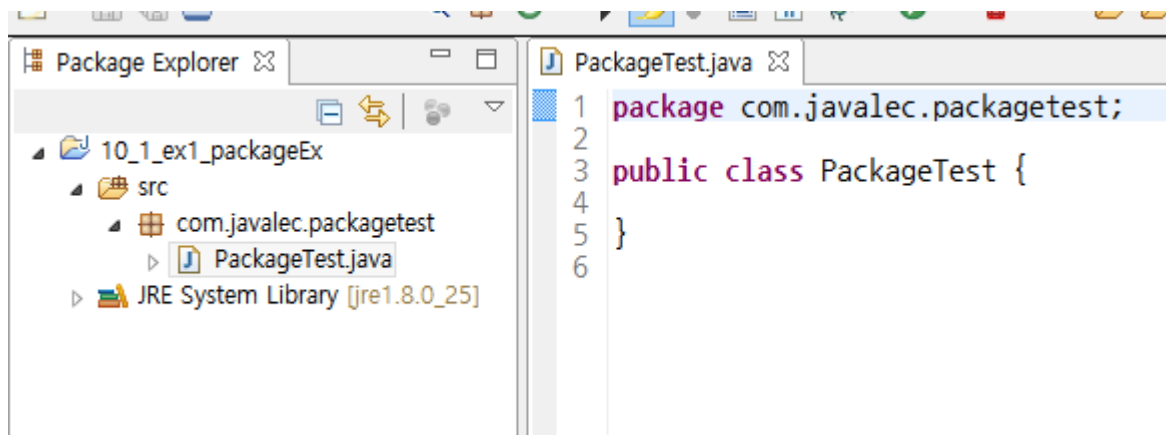
JAVA API는 다운받아서 자주 보는 습관을 들이면 좋습니다. 자바 사전이라고 생각하시면 됩니다.

10-1. 패키지의 이해

우리가 만드는 패키지

JAVA에 패키지가 있듯이 우리도 클래스를 만들 때 패키지로 관리하면 효율적인 작업을 할 수 있습니다.
이제 직접 패키지를 만들어 보도록 하겠습니다.

예제 : 10_1_ex1_packageEx



10-2. 정답은 없지만, 효율적으로 해야 하는 패키지의 일반적인 설계 방식

정답은 없지만, 효율적으로 해야 하는 패키지의 일반적인 설계 방식

초보 개발자 뿐만이 아니라, 오래된 개발자의 경우에도 패키지를 만들 때 패키지 이름에 대해서 다소 스트레스를 받는 경향이 있습니다.

패키지 이름은 유일한 이름일수록 좋습니다. 이유는 우리회사에서 개발은 소스(.java파일)를 다른 곳에 이식하려 할 때 이름이 중복되면 좋지 않기 때문 입니다. 그래서 주로 패키지 이름은 유니크한 도메인 주소를 역방향으로 하여 만듭니다.

예를 들어 회사 도메인이 www.javalec.com 이라면, 'com.javalec.'라고 하고 뒤에 기능별 또는 개발팀에서 정한 규칙에 의해 패키지명을 만듭니다.

물론 패키지명은 개발 중 언제든지 수정이 가능합니다.

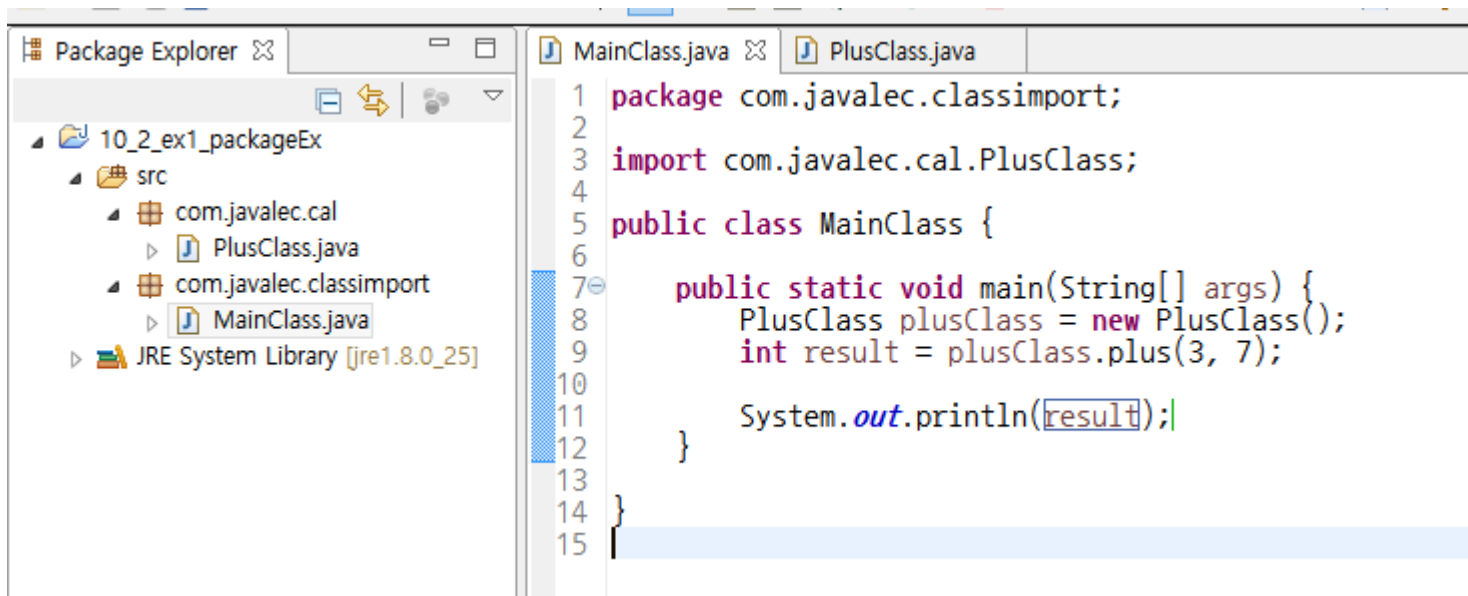
따라서 패키지명을 만들 때 너무 스트레스를 받지 말고, 쉽게 쉽게 생각하면 될 거 같습니다.

10-3. import의 이해

다른 패키지의 클래스를 가져오자~

패키지를 만들고 클래스를 만든 후 서로 다른 패키지의 클래스를 사용하려고 할 때는 반드시 import 키워드를 사용해서 해당 클래스를 임포트 해야 합니다. 하지만 우리가 사용하는 이클립스가 자동으로 임포트를 해주기 때문에 번거로운 거는 하나도 없습니다.

예제 : 10_2_ex1_packageEx



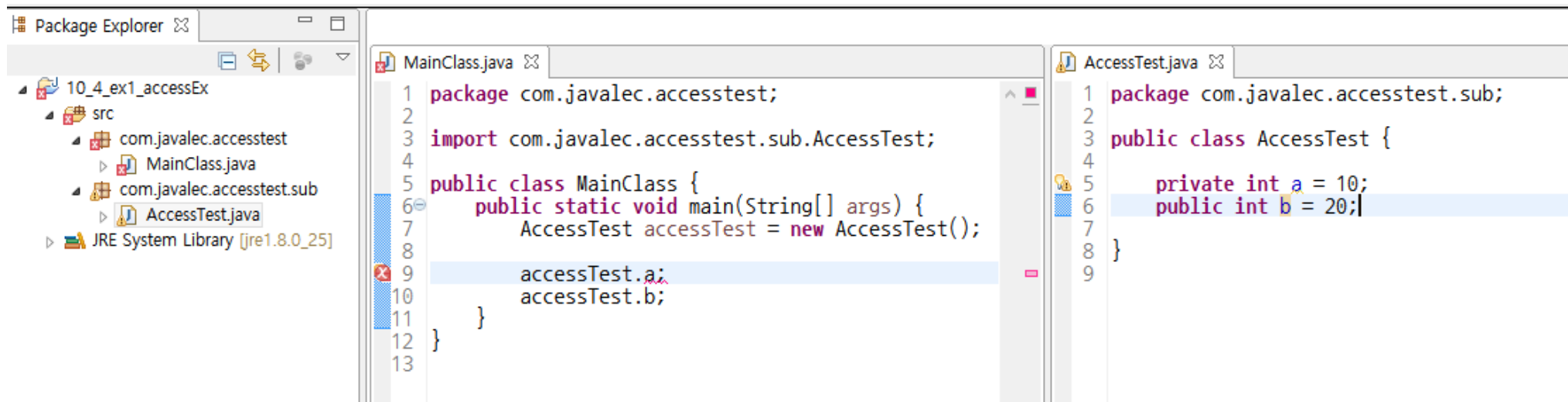
10-4. 접근 제한 이란?

제한 합니다.

접근 제한이란, 말 그대로 클래스의 데이터(인스턴스) 또는 메소드에 대해서 다른 클래스로부터의 접근을 제한하는 것 입니다.

즉 다른 클래스의 어떤 데이터가 접근을 제한하고 있다면, 그 데이터는 외부 다른 클래스에는 사용을 할 수가 없는 것 입니다.

예제 : 10_4_ex1_accessEx



```
Package Explorer
├── 10_4_ex1_accessEx
│   ├── src
│   │   ├── com.javalec.accesstest
│   │   │   ├── MainClass.java
│   │   └── com.javalec.accesstest.sub
│   │       ├── AccessTest.java
│   └── JRE System Library [jre1.8.0_25]
MainClass.java
1 package com.javalec.accesstest;
2
3 import com.javalec.accesstest.sub.AccessTest;
4
5 public class MainClass {
6     public static void main(String[] args) {
7         AccessTest accessTest = new AccessTest();
8
9         accessTest.a;
10        accessTest.b;
11    }
12 }
13
AccessTest.java
1 package com.javalec.accesstest.sub;
2
3 public class AccessTest {
4
5     private int a = 10;
6     public int b = 20;
7
8 }
9
```

10-5. 접근 제한의 종류

접근 제한 종류

- public : 접근 제한을 하지 않습니다. 즉 다른 모든 클래스에서 사용 가능 합니다.
- private : 자신의 클래스에서만 사용 가능 합니다. 외부에서는 사용할 수 없습니다.
- protected : 해당 클래스와 동일한 패키지에 있거나 추후에 배울 상속받은 클래스일 경우에만 사용 가능 합니다.
- default : 해당 클래스와 동일한 패키지에서만 사용 가능 합니다.

위4가지 중에서 public 과 private이 가장 많이 사용하게 됩니다.



static

- static의 이해
- static과 객체와의 관계
- static의 장점과 단점

11-1. static의 이해

static 키워드의 이해

필요에 의해 클래스를 만들고, 사용할 때는 생성자를 통해 객체로 만들어서 사용을 합니다.
이러한 방식으로 모든 것을 처리하다 보면, 문제가 생길 수 있습니다.

예를 들어 집안에 아빠, 엄마, 그리고 세 쌍둥이가 있습니다.
아이들이 아빠 지갑에 100원짜리 2개가 있다고 알고 있습니다.
이러한 상황에서 아이들이 아빠한테 100원을 달라고 합니다.
당연히 아빠지갑에는 100원 동전 2개가 있으니, 줄 수 있습니다.

첫째 아이가 먼저 아빠 지갑 객체를 생성하여 100원을 받았습니다.
그리고 둘째 아이가 아빠 지갑 객체를 생성하여 100원을 받았습니다.
마지막으로 셋째 아이가 아빠 지갑 객체를 생성하여 100원을 받았습니다.

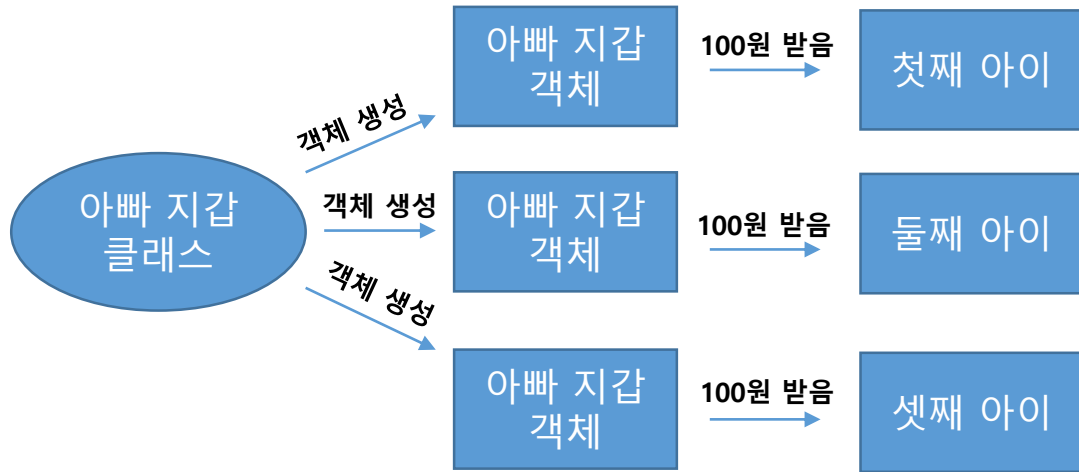
여기까지 보면, 뭔가 이상 합니다. 분명 집안에 아빠도 한 명이고, 지갑도 한 개일 텐데, 아이들 모두 아빠 지갑을 생성하여 100원씩을 받았습니다.
200원 밖에 없는데 아이들이 받은 돈은 모두 300원 입니다. 버그 입니다.

그럼 이런 상황에서 생각할 수 있는 방법은 무엇일까요?
아주 쉽게 생각해 보면 아빠 지갑 객체가 세 아이들 모두에게 공유 되면 됩니다.
그러면 첫째 아이가 100원 받고, 둘째 아이가 100원을 받으면, 지갑에는 동전이 없고, 셋째 아이는 100원을 받을 수 없게 됩니다.
셋째 아이한테는 서운한 일이지만, 어쨌든 프로그램적으로는 이상이 없이 정상 동작 할 것입니다.

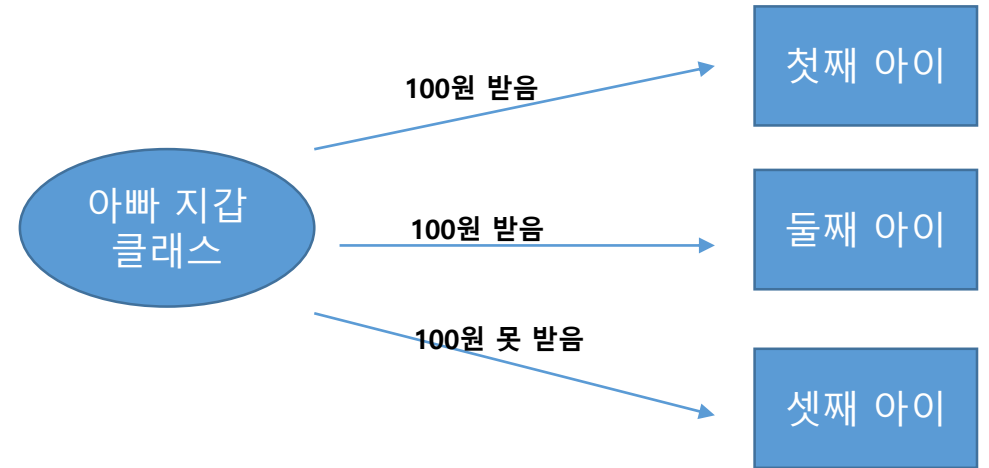
그러면 어떻게 아빠의 지갑을 모든 아이들이 공유할 수 있을까요?
바로 'static' 키워드를 사용하는 것 입니다.
우선 'static' 키워드를 '공유' 라고 생각 합니다.

11-1. static의 이해

static 키워드의 이해



현실적으로 불가능

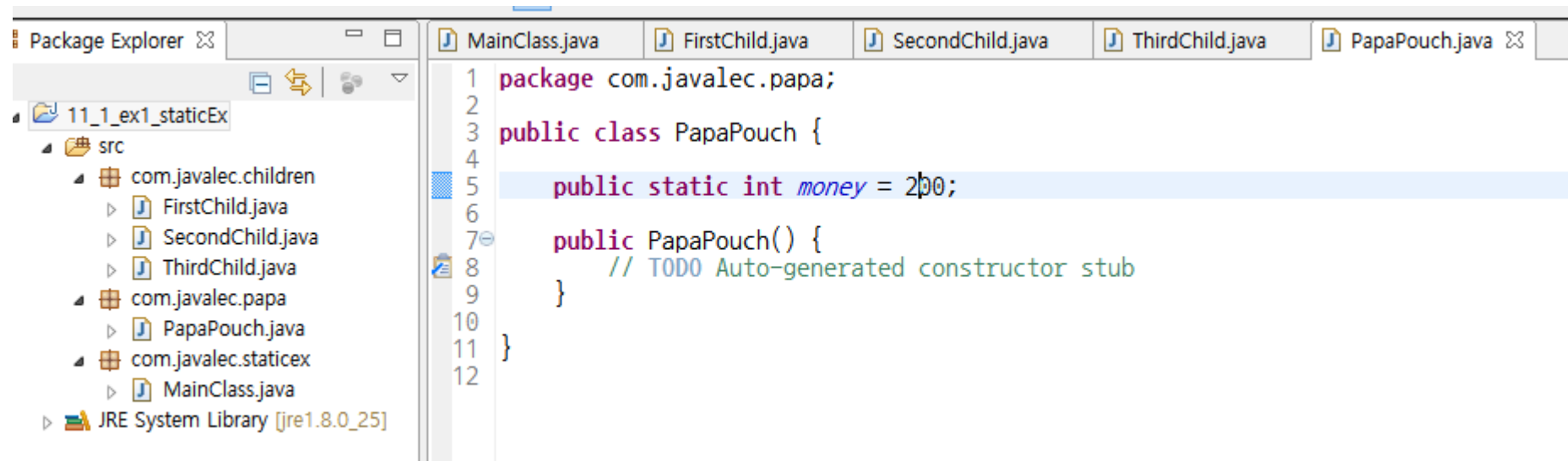


현실적으로 가능

11-1. static의 이해

static 키워드의 이해

위와 같은 상황을 'static' 키워드를 사용하여 프로그래밍 해 보겠습니다.(11_1_ex1_staticEx)



11-2. static과 객체와의 관계

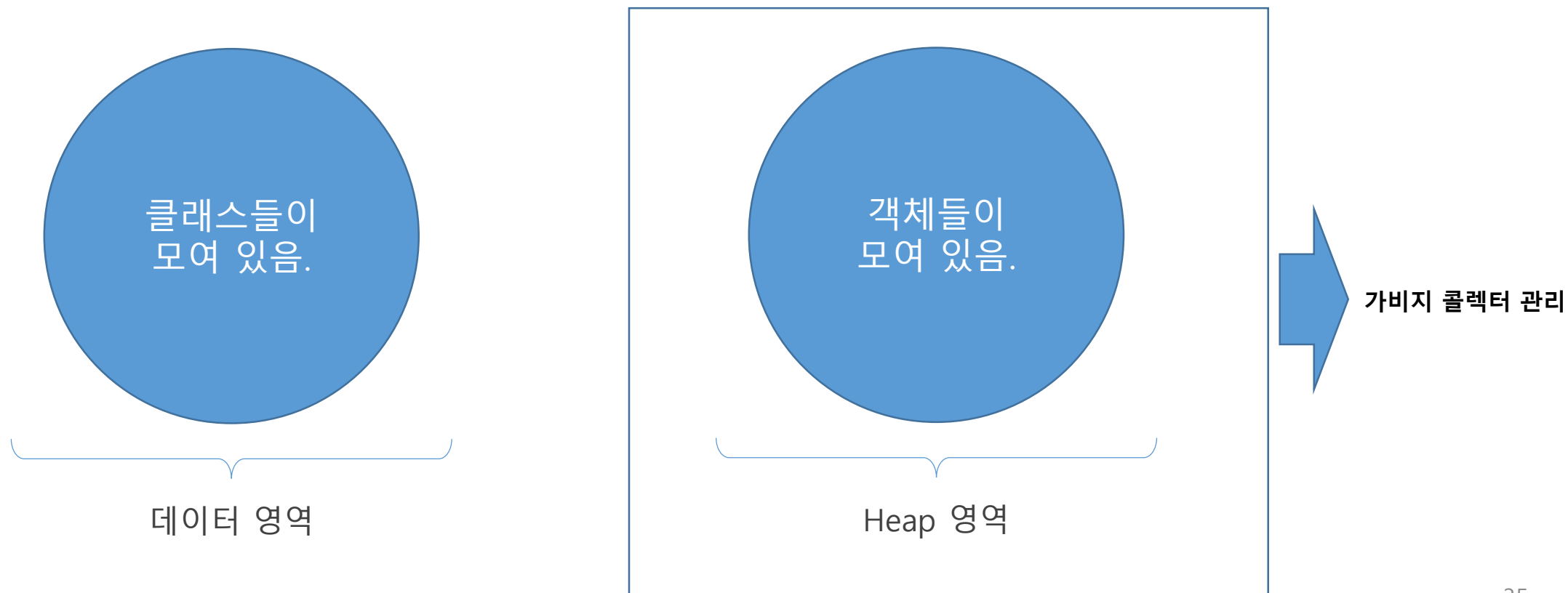
static은 클래스 변수

static 키워드를 사용한 변수는 클래스 변수입니다.

객체는 클래스에서 생성(복사)되어 진다고 배웠습니다.

클래스 하나에서 무한대로 객체를 생성할 수 있고 그러한 객체는 모양은 동일할지 모르지만, 전혀 다른 덩어리 입니다.

하지만 static이 붙은 변수는 객체 변수가 아닌 클래스 변수로써 객체가 생성되기 전에 이미 존재합니다.



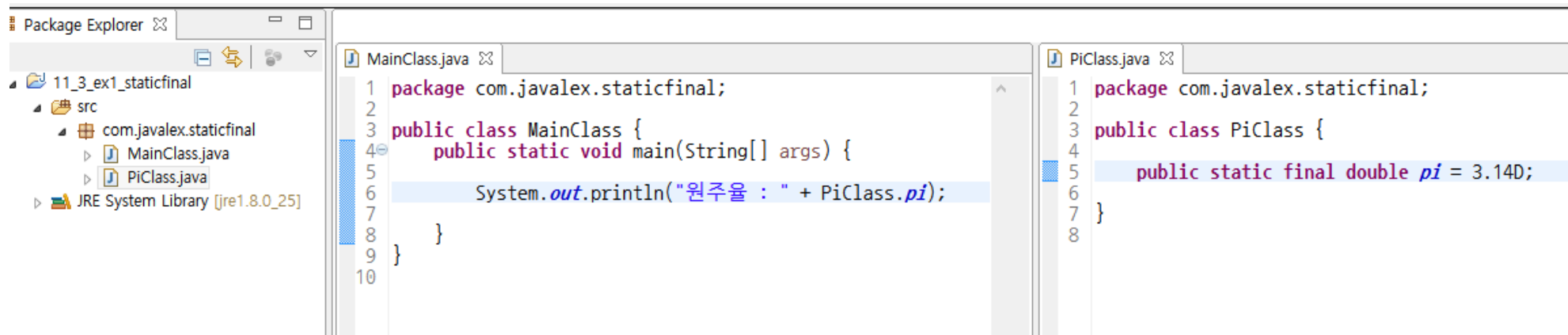
11-3. static의 장점과 단점

static은 잘 사용하면 좋고, 그렇지 않으면 큰 재앙을 불러 올 수 있습니다.

static 키워드를 사용한 변수는 객체생성과는 상관없이 클래스와 함께 존재 하게 됩니다. 즉 객체 생성을 하지 않아도 메모리에 상주하게 됩니다. 이렇게 생성을 하지 않고도 존재한다는 것은 그만큼 메모리를 사용하지 않아도 된다는 것 입니다. 지금은 간단한 예제를 살펴보고 있기 때문에 객체생성에 따른 메모리의 부하를 느낄 수 없지만, 프로젝트가 커지면 객체생성에 따른 메모리 부족 현상이 있을 수 있습니다. 물론 가비지컬렉터가 열심히 일을 하고는 있지만요. 객체 생성을 하지 않고도 사용할 수 있다는 것은 분명 장점이지만, 한편으로 생각해 보면 가비지컬렉터의 관리 밖에 있기 때문에 항상 메모리에 상주해 있습니다. 즉 프로젝트가 커지고, 시스템이 오랜 시간 동안 돌아가게 되면 시스템 운영속도가 점차 느려지다가 급기야 큰 재앙이 올 수도 있습니다. 따라서 static의 사용은 신중해야 됩니다.

static 변수로 사용되는 대표적인 예를 아마도 상수(변하지 않는 수) 일 것입니다. 만약 프로그래밍을 하다가 원주율이 빈번이 사용되어야 한다면 이러한 값은 상수도 정의해 놓고 사용하는 편이 좋습니다.

원주율은 3.141592653589793238462643383279... 끝이 없습니다.
이러한 경우 원주율을 3.14까지로 정의해 놓고 사용하자고 팀원들 사이 약속을 합니다.





상속 - I

- 상속! 어렵지 않아요. 일상에서 알고 있는 상속입니다.
- 상속이 필요한 이유
- 상속 문법의 이해
- 상속 예제 - 대대로 이어오고 있는 식당 메뉴

12-1. 상속! 어렵지 않아요. 일상에서 알고 있는 상속입니다.

아들왈 : 아버지~~ 편하게 살게 재산 많이 물려 주세요.

객체지향 언어의 대표적인 JAVA를 공부 할 때 가장 중요하고 꼭 알고 넘어가야 하는 부분이 상속과 추상화 개념입니다.

이렇게 중요한 부분을 정확하게 알고 사용하는 개발자는 의외로 많지 않습니다.

그런 개발자를 누군가는 이렇게 말합니다. '무늬만 개발자'라고요.

다소 농담 섞인 의견 이었으나, 중요한 것은 객체지향 언어를 하면서 상속과 추상화를 모르면 객체지향 언어를 한다고 말할 수 없을 정도로 상속과 추상화 개념은 중요 합니다. 하지만 절대로 어렵지 않습니다. 아주 쉽습니다.

상속은 우리가 일상에서 알고 있는 상속 입니다. 아버님의 재산을 물려받는 아들의 경우 아버님으로부터 아들은 재산을 상속 받은 경우 입니다.

반대로 아버님이 많은 빚을 아들에게 물려주는 경우도 있습니다. 이 경우에도 아들은 아버님으로부터 많은 빚을 상속 받은 경우 입니다.

프로그래밍에서도 어떤 객체가 다른 객체로부터 객체의 데이터(속성)와 메소드를 상속 받을 수 있습니다. 이럴 경우 상속받은 객체는 상속해준 객체의 데이터 및 메소드를 이용할 수 있고 또는 변경 할 수도 있습니다.

상속을 통해서 4발 자전거를 만든다고 생각해 보겠습니다.

자전거에는 기본적으로 핸들, 바퀴, 브레이크, 페달, 프레임 등이 있어야 합니다.

그리고 4발 자전거는 바퀴가 4개 있어야 합니다. 자전거 제작자가 처음부터 4발 자전거를 만들려고 하면 많은 시간과 노력이 필요 합니다.

그래서 자전거 제작자는 아이디어를 생각해 냅니다. 자전거 가게에 가서 2발 자전거를 사가지고 와서, 뒷바퀴에 2개의 바퀴를 달면 4발 자전거가 쉽게 제작 되어집니다. 몇 일이 걸릴 수 있고, 여러 번의 시행착오가 생길 수 있는 자전거 제작을 단 몇 분만에 똑딱 끝낼 수 있게 되었습니다.

프로그램도 마찬가지 입니다. 기존의 좋은 클래스가 있다면 상속을 통해서 개발시간을 단축시킬 수 있습니다.



상속

상속하고, 필요한 것은 추가하고, 변경할
곳은 변경하였습니다.



12-2. 상속이 필요한 이유

항상 맨땅에 헤딩 할 수는 없습니다. 지름길이 있다면 쉽게 갈 수도 있습니다.

상속이 필요한 이유는 크게 두 가지 이유입니다.

첫째로는, 모든 개발을 처음부터 하얀 도화지에 할 필요는 없습니다. 이미 훌륭한 선배님들께서 만들어 놓은 프로그램이 있다면 상속을 통해서 훌륭한 결과물을 빠른 시간 내에 만들 수 있습니다. 또한 기존의 훌륭한 프로그램은 대부분 검증이 잘 되어 있어 버그도 거의 없을 수 있습니다.

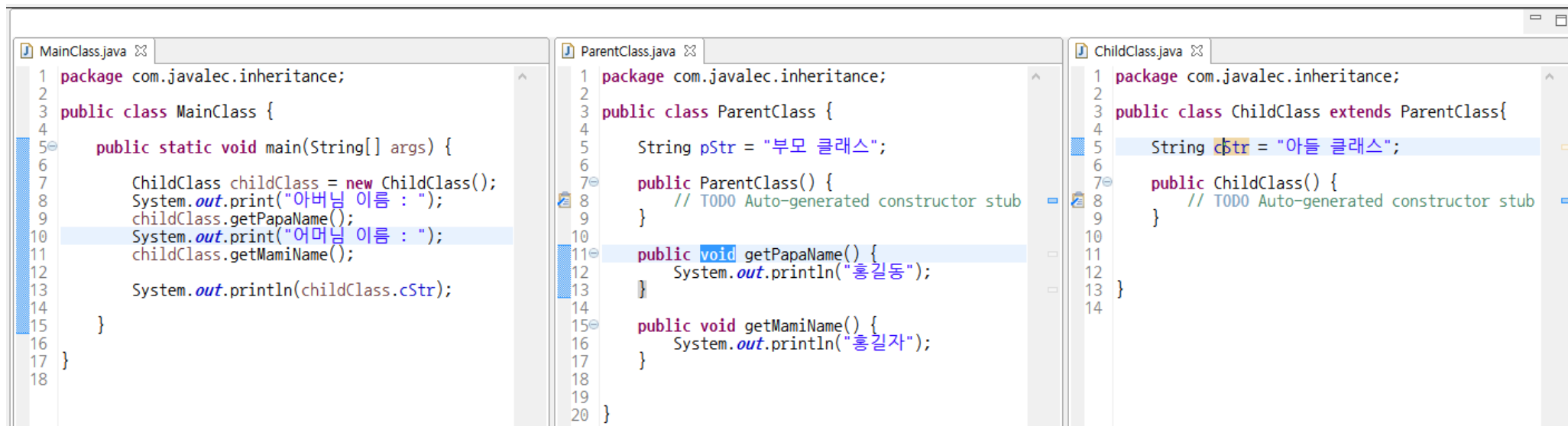
둘째로는, **다양한 객체(타입)를 상속을 통해서 하나의 객체(타입)으로 묶을 수 있습니다.**

이것은 추상화 개념의 이해가 있어야 하는데요, 아직은 추상화에 대해서 살펴보지 않았으므로 우선은 다양한 객체를 하나로 통일시킬 수 있다고만 알고 있으면 될 거 같습니다.

12-3. 상속 문법의 이해

프로그램에서 상속의 문법을 배워 보자.

상속의 개념에 대해서 이제 감을 잡았을 거 같습니다. 사실 별로 어려운 부분도 없었습니다. 상속은 말 그대로 상속일 뿐입니다.



```
1 package com.javalec.inheritance;
2
3 public class MainClass {
4
5     public static void main(String[] args) {
6
7         ChildClass childClass = new ChildClass();
8         System.out.print("아버님 이름 : ");
9         childClass.getPapaName();
10        System.out.print("어머님 이름 : ");
11        childClass.getMamiName();
12
13        System.out.println(childClass.cStr);
14
15    }
16 }
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2
```


12-4. 상속 예제 – 대대로 이어오고 있는 식당 메뉴

시대가 변하면 메뉴도 업그레이드 되어야 합니다.(12_4_ex1_menu)

대대로 이어져오는 식당이 있습니다. 이식당은 변하지 않는 메뉴도 있지만, 시대의 변화에 맞춰 새로운 메뉴가 생겼습니다.

청국장
된장국
갈비찜
콩비지

예전 메뉴

청국장
쇠고기 청국장
된장국
얼큰 된장국
콩비지
콩나물국

현재 메뉴

12-4. 상속 예제 – 대대로 이어오고 있는 식당 메뉴

시대가 변하면 메뉴도 업그레이드 되어야 합니다.

```
MainClass.java
1 package com.javalec.mune;
2
3
4 public class MainClass {
5     public static void main(String[] args) {
6
7         ChildMenu childMenu = new ChildMenu();
8         childMenu.makeChungGukJang();
9         childMenu.makeDoenJangGuk();
10        childMenu.makeGalbiljim();
11        childMenu.makeHotDoenJangGuk();
12        childMenu.makeKongNaMool();
13        childMenu.MakeSoybean();
14    }
15 }
16 }
17 }

ParentMenu.java
1 package com.javalec.mune;
2
3 public class ParentMenu {
4
5     public ParentMenu() {
6         // TODO Auto-generated constructor stub
7     }
8
9     public void makeChungGukJang() {
10        // TODO Auto-generated method stub
11        System.out.println("청국장");
12    }
13
14    public void makeDoenJangGuk() {
15        // TODO Auto-generated method stub
16        System.out.println("된장국");
17    }
18
19    public void makeGalbiljim() {
20        // TODO Auto-generated method stub
21        System.out.println("갈비찜");
22    }
23
24    public void MakeSoybean() {
25        // TODO Auto-generated method stub
26        System.out.println("콩비지");
27    }
28 }
29 }

ChildMenu.java
1 package com.javalec.mune;
2
3 public class ChildMenu extends ParentMenu {
4
5     public ChildMenu() {
6         // TODO Auto-generated constructor stub
7     }
8
9     private void makeBeefChungGukJang() {
10        // TODO Auto-generated method stub
11        System.out.println("쇠고기 청국장");
12    }
13
14    public void makeHotDoenJangGuk() {
15        // TODO Auto-generated method stub
16        System.out.println("얼큰 된장국");
17    }
18
19    public void makeKongNaMool() {
20        // TODO Auto-generated method stub
21        System.out.println("콩나물국");
22    }
23
24    @Override
25    public void makeChungGukJang() {
26        // TODO Auto-generated method stub
27        System.out.println("냄새 없는 청국장");
28    }
29 }
30 }
```



상속 - II

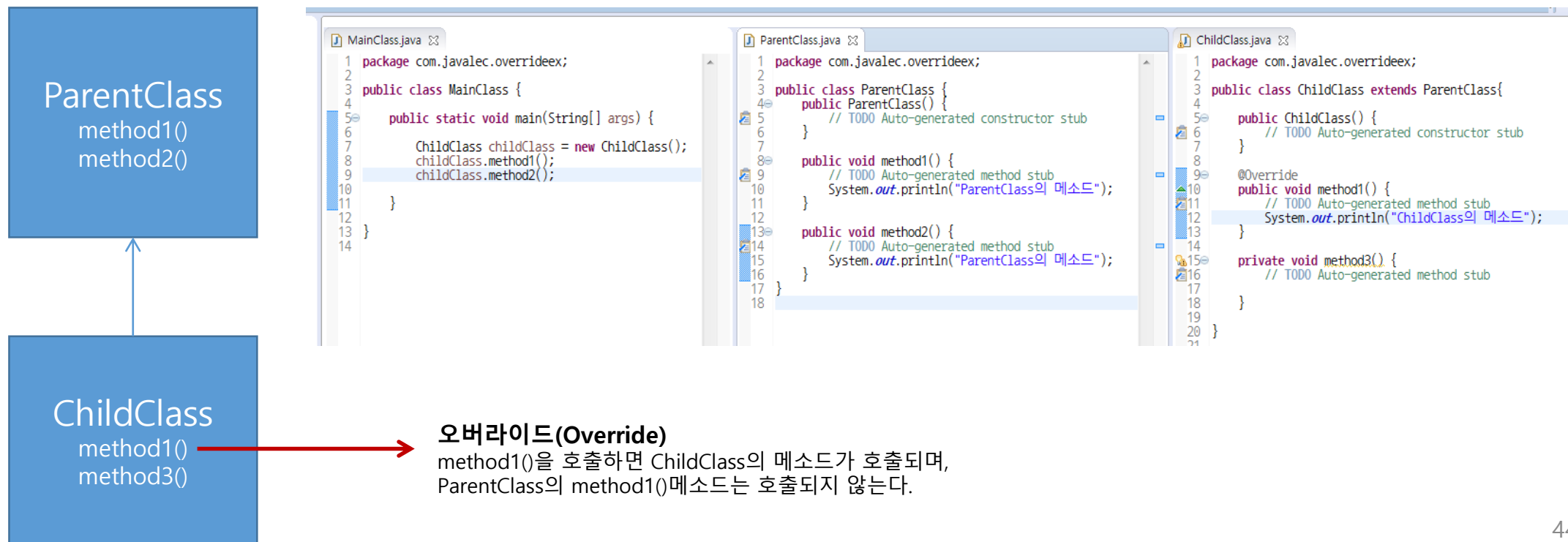
- 오버라이드의 이해
- 체인점 사업을 통한 상속의 예
- super키워드(오버라이드의 장점과 단점의 이해)

13-1. 오버라이드의 이해

오버라이드(Override) - 부모 클래스의 메소드를 자식 클래스에서 재정의 함.

자식 클래스가 부모 클래스를 상속하여 자식한테 없는 메소드를 호출하면 부모클래스에 가서 해당 메소드를 찾게 됩니다.

만약 부모 클래스의 메소드를 자식클래스에서 동일한 이름으로 다시 재정의 하면 부모클래스의 메소드를 찾지 않고 자식 클래스의 메소드를 호출하게 됩니다.
이것을 오버라이드(재정의)라고 합니다.



13-2. 체인점 사업을 통한 상속의 예

본사의 메뉴 가격을 기본적으로 따르되, 지역 특색에 맞춰 가격을 변경 함

본사

김치찌개 – 5,000원
부대찌개 – 6,000원
비빔밥 – 6,000원
순대국 – 5,000원
공기밥 – 1,000원

주택가에 매장1호점

김치찌개 – 4,500원
부대찌개 – 5,000원
비빔밥 – 6,000원
순대국 – 판매하지 않음
공기밥 – 1,000원

대학가에 매장2호점

김치찌개 – 5,000원
부대찌개 – 5,000원
비빔밥 – 5,000원
순대국 – 4,000원
공기밥 – 무료

증권가에 매장3호점

김치찌개 – 6,000원
부대찌개 – 7,000원
비빔밥 – 7,000원
순대국 – 6,000원
공기밥 – 1,000원

```
1 package com.javalec.store;
2
3 public class StoreNum2 extends HeadQuarterStore {
4
5     public StoreNum2() {
6         // TODO Auto-generated constructor stub
7     }
8
9     @Override
10    public void orderBuDaejiige() {
11        // TODO Auto-generated method stub
12        System.out.println("5,000원 입니다.");
13    }
14
15    @Override
16    public void orderBiBimbap() {
17        // TODO Auto-generated method stub
18        System.out.println("5,000원 입니다.");
19    }
20
21    @Override
22    public void orderSunDaeguk() {
23        // TODO Auto-generated method stub
24        System.out.println("4,000원 입니다.");
25    }
26
27    @Override
28    public void orderGongGibap() {
29        // TODO Auto-generated method stub
30        System.out.println("무료 입니다.");
31    }
32
33 }
34
35
36 package com.javalec.store;
37
38 public class HeadQuarterStore {
39
40     public HeadQuarterStore() {
41     }
42
43     public void orderKimChijiige() {
44         System.out.println("5,000원 입니다.");
45     }
46
47     public void orderBuDaejiige() {
48         System.out.println("6,000원 입니다.");
49     }
50
51     public void orderBiBimbap() {
52         System.out.println("6,000원 입니다.");
53     }
54
55     public void orderSunDaeguk() {
56         System.out.println("5,000원 입니다.");
57     }
58
59     public void orderGongGibap() {
60         System.out.println("1,000원 입니다.");
61     }
62 }
63
64 package com.javalec.store;
65
66 public class StoreNum1 extends HeadQuarterStore {
67
68     public StoreNum1() {
69         // TODO Auto-generated constructor stub
70     }
71
72     @Override
73     public void orderKimChijiige() {
74         System.out.println("4,500원 입니다.");
75     }
76
77     @Override
78     public void orderBuDaejiige() {
79         System.out.println("5,000원 입니다.");
80     }
81
82     @Override
83     public void orderSunDaeguk() {
84         System.out.println("판매 하지 않습니다.");
85     }
86 }
87
88 }
```

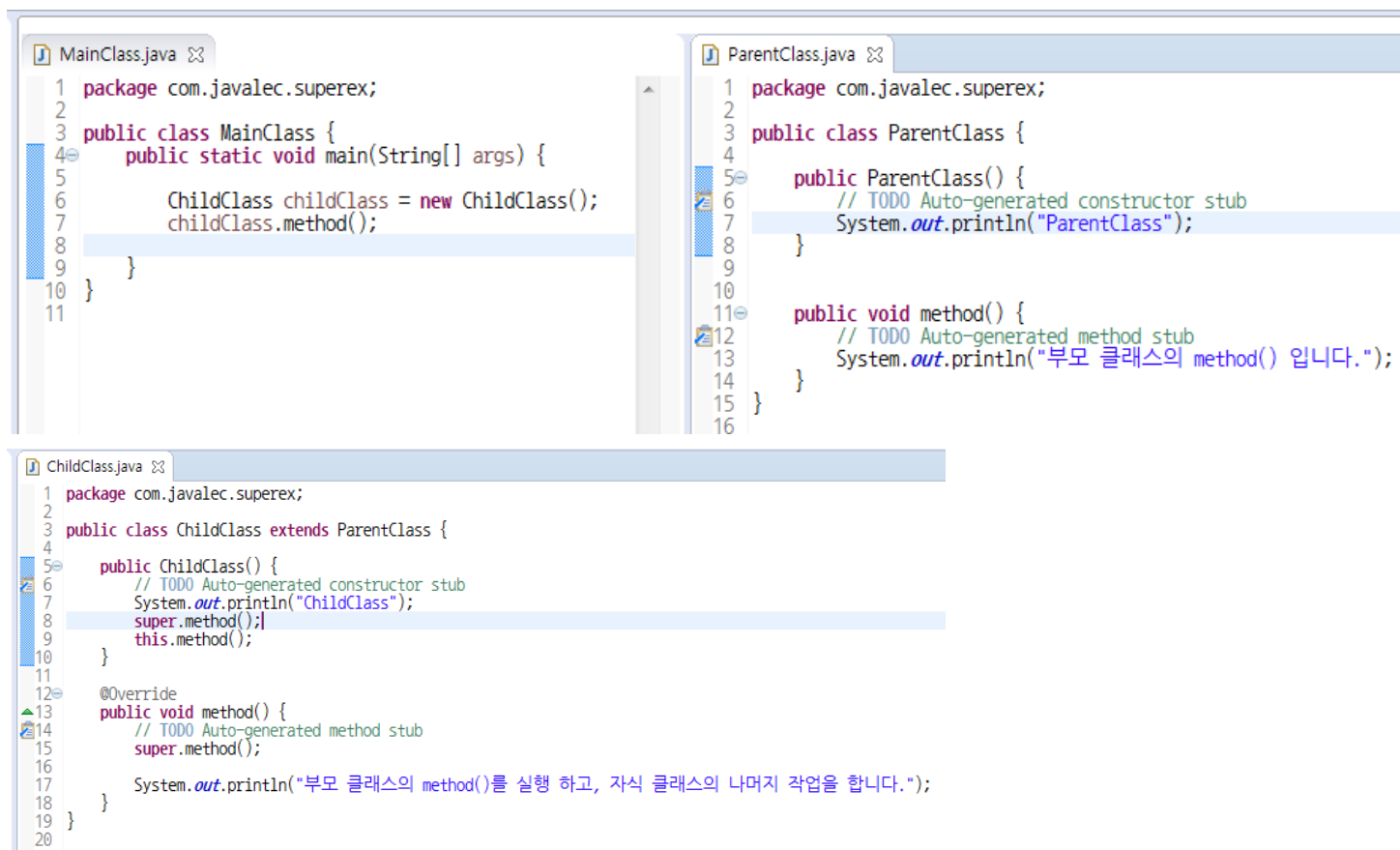
13-3. super 키워드(오버라이드의 장점과 단점의 이해)

super는 부모를 찾는 키워드

override의 장점은 잘 만들어진 클래스를 상속받아 일부 메소드를 수정(재정의) 할 수 있다는 것 입니다.

하지만, 자식클래스에서 부모클래스를 override하면 부모클래스의 메소드는 사용할 수가 없습니다. 이것이 override의 단점이라고 할 수 있습니다.

자식 클래스에서 부모클래스의 메소드를 사용하고자 할 때 super키워드를 사용 합니다.



```
1 package com.javalec.superex;
2
3 public class MainClass {
4     public static void main(String[] args) {
5
6         ChildClass childClass = new ChildClass();
7         childClass.method();
8     }
9 }
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2
```